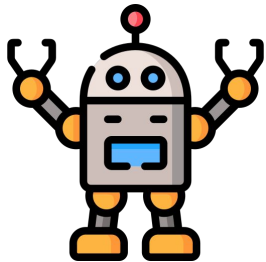


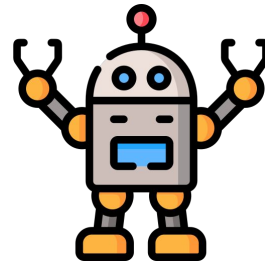
BLACK JACK

Reinforcement Learning for Blackjack

using Q-Learning



Why Reinforcement Learning?



- Blackjack is a **sequential decision-making process** — every action influences the next state.
- The environment is **stochastic** (cards are random, outcome is uncertain).
- The agent doesn't have full knowledge of the environment — a perfect case for **model-free RL**.
- The goal can be expressed as **maximizing long-term reward** (winning probability).

You can't predefine “correct answers” for every card combination — they must be *learned through interaction*.

Theory: Q-learning

Main idea:

Q-Learning is a fundamental RL algorithm that builds a table of **Q-values** — expected rewards for taking action a in state s .

Over many iterations, the Q-table converges toward the optimal strategy.

After training, the algorithm will make decisions (choose actions) based on the best reward for the current state in the table.

State	stand	hit
S1	-1	0.88
S2	0.2	0.8
S3	-1.7	1.2
...	...	...

State format: (player_sum, dealer_card, usable_ace, last_card)

How to update values in Q-values table?

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Q(s, a) - expected rewards for taking action a in state s.

max Q(s', a') - max possible Q value on the next step

[r+γmaxQ(s',a')-Q(s,a)] - value which we get (observed) minus expected value(in table)

α - learning rate (how quickly we update our knowledge)

γ - discount factor (importance of future rewards)

r - immediate reward from the environment,

s' - next state after taking action.

a' - next state possible action after taking current action



Blackjack

Each player and the dealer start with two cards.

The goal: reach a sum as close to 21 as possible without exceeding it.

Card values:

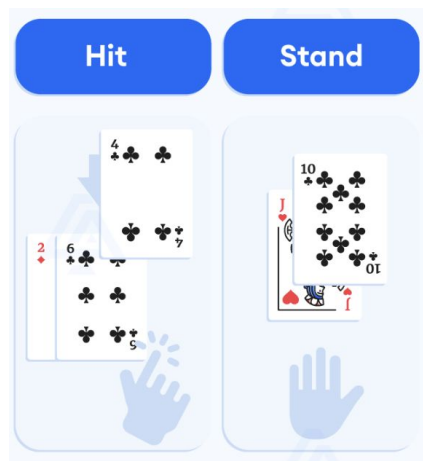
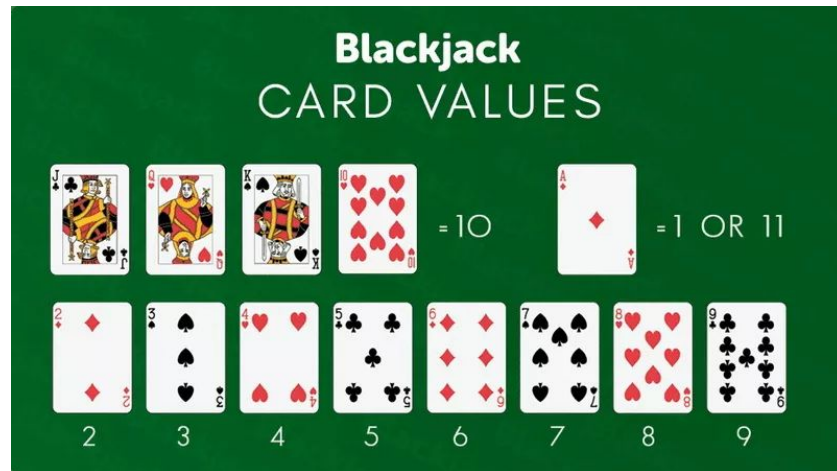
- 2-10: face value
- J, Q, K: 10
- Ace: 1 or 11

Player actions:

- Hit → take another card
- Stand → stop taking cards

Dealer hits until reaching at least 17.

This setup provides a simple and clear environment for RL training.



Real Statistical Advantage of the Dealer

The dealer acts **after** the player, giving the house an advantage.

Even with perfect play, the **player wins only about 42–44%** of games.

The **dealer wins around 49%**, and **draws occur about 9%** of the time.

This means the environment is **biased against the player**, and any strategy that improves on the dealer's baseline performance is already a success.



How all works in my case?

In this project, Q-Learning is used to learn the best decision (“hit” or “stand”) for each possible situation.

- **The environment provides the agent with:**
- **The current sum of the player’s cards.**
- **The dealer’s visible card.**
- **Whether there is a usable Ace.**
- **The last card drawn.**

The agent receives a reward of:

- +1 if it wins,
- 0 for a draw,
- -1 for a loss

Training:

- using 300 000 000 games simulations for training

State	stand	hit
S1	-1	0.88
S2	0.2	0.8
S3	-1.7	1.2
...	...	...

State format: (player_sum, dealer_card, usable_ace, last_card)

Each entry in the Q-table looks like this:

$Q[(\text{player_sum}, \text{dealer_card}, \text{usable_ace}, \text{last_card})] = [Q(\text{stand}), Q(\text{hit})]$

This means:

For each unique game state, the agent stores expected rewards for both actions.

During training, values are updated according to actual rewards.

During testing, the agent simply chooses the action with the higher Q-value.

This allows the model to act without randomness after training.

Metrics

As a final metric for evaluation, I decided to use the percentage of wins, losses, and draws.

For clarity, I took into account draws and no draws.

Results on 100 000 games:



```
statistic with draft:
```

```
win games in %: 0.45343, lose games in %: 0.46081, draw games in %: 0.08576
```

```
statistic without draft:
```

```
win games in %: 0.49596386069303466, lose games in %: 0.5040361393069653
```


Compare with Dealer Agent

Dealer-type agent — follows the rule “hit until 17”

Results on 100 000 games:

Dealer Agent strategy:

```
statistic with draft:  
win games in %: 0.40299, lose games in %: 0.50169, draw games in %: 0.09532  
statistic without draft:  
win games in %: 0.4454503249767874, lose games in %: 0.5545496750232126
```

Q learning Agent strategy:

```
statistic with draft:  
win games in %: 0.45343, lose games in %: 0.46081, draw games in %: 0.08576  
statistic without draft:  
win games in %: 0.49596386069303466, lose games in %: 0.5040361393069653
```



Results and Conclusions

- The agent successfully learned to play Blackjack through experience.
- Q-Learning effectively adapted to the environment's randomness.
- The learned strategy performs better than the dealer rule.
- Even with the casino's natural advantage, the RL agent achieved a statistically higher win rate.